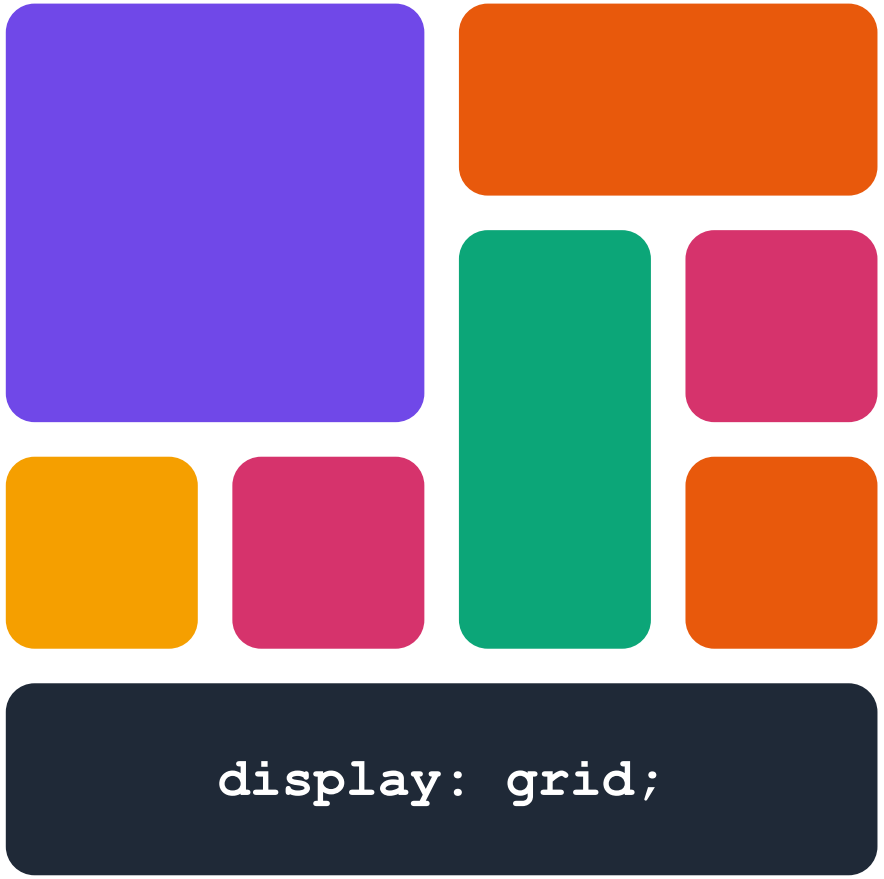
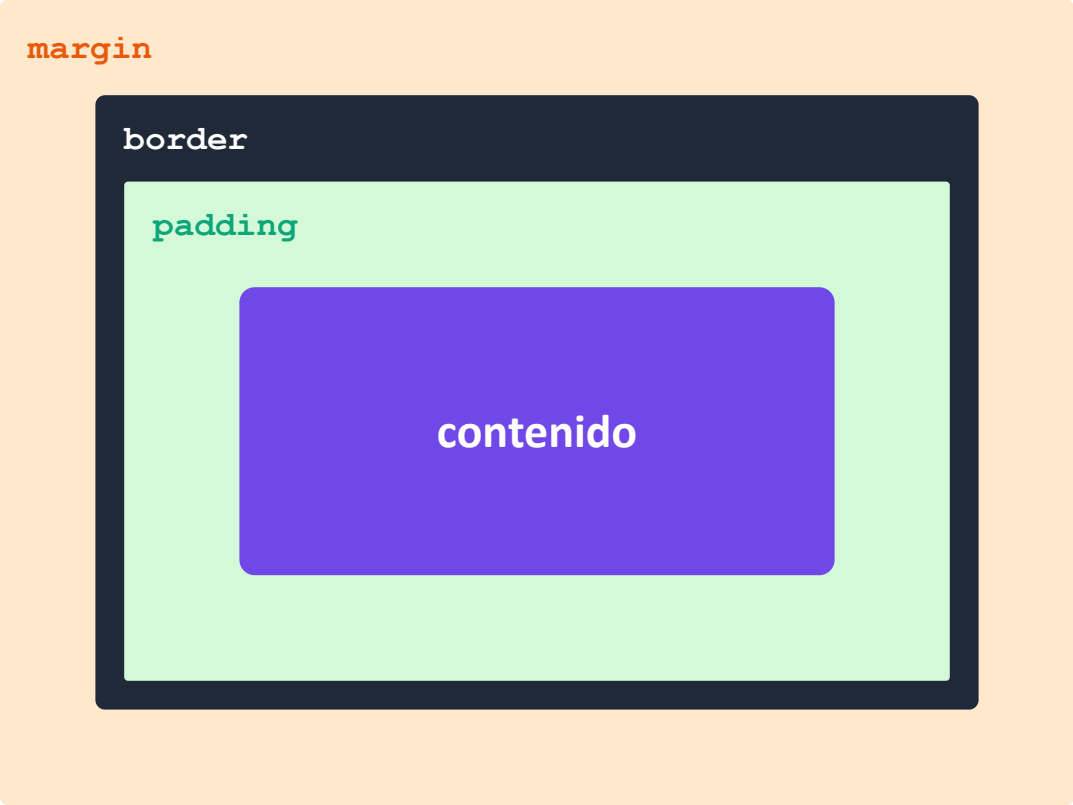
Layout en CSS

Flujo normal, float, Flexbox y Grid

Tema 4



Todo es una caja, un contenedor



Cada elemento HTML genera un rectángulo. **Hacer layout es decidir dónde va cada rectángulo** respecto a los demás.

Tres herramientas para colocarlos:

- **Flujo normal**: lo que hace el navegador si no le dices nada.
- **Flexbox**: filas o columnas (1 dimensión).
- **Grid**: filas y columnas a la vez (2 dimensiones).

```css
/* width incluye padding y borde */
* { box-sizing: border-box; }
```

01

Flujo normal, display y float

Cómo coloca el navegador las cosas si no le decimos nada

Flujo normal: bloques que se apilan, texto que fluye



Sin CSS de layout el navegador ya organiza la página:

- Los elementos **block** (`div`, `p`, `h1`, `section`) ocupan todo el ancho y se apilan de arriba abajo.
- Los elementos **inline** (`a`, `span`, `strong`) van dentro de la línea de texto, de izquierda a derecha.
- Cuando cambiamos el layout con CSS, lo que hacemos es **sacar** elementos de este flujo.

```
<div class="block">Elemento 1</div>
<div class="block">Elemento 2</div>
<div class="inline">Elemento 3</div>
```

display: block, inline e inline-block

```css
.block {
  display: block;  /* fila entera */
}
.inline {
  display: inline;  /* como una palabra */
  width: 300px;  /* IGNORADO */
}
.inline-block {
  display: inline-block;  /* en línea + tamaño */
  width: 160px;
}
```

- Flexbox y Grid son **dos valores más** de esta misma propiedad: `display: flex` y `display: grid`.

block

Bloque 1

Bloque 2

inline

Texto **inline A** sigue el texto **inline B** y continúa.

inline-block

Texto **160px** sigue **160px**

Ejemplo 01 · 01-display.html

Comparativa rápida de display

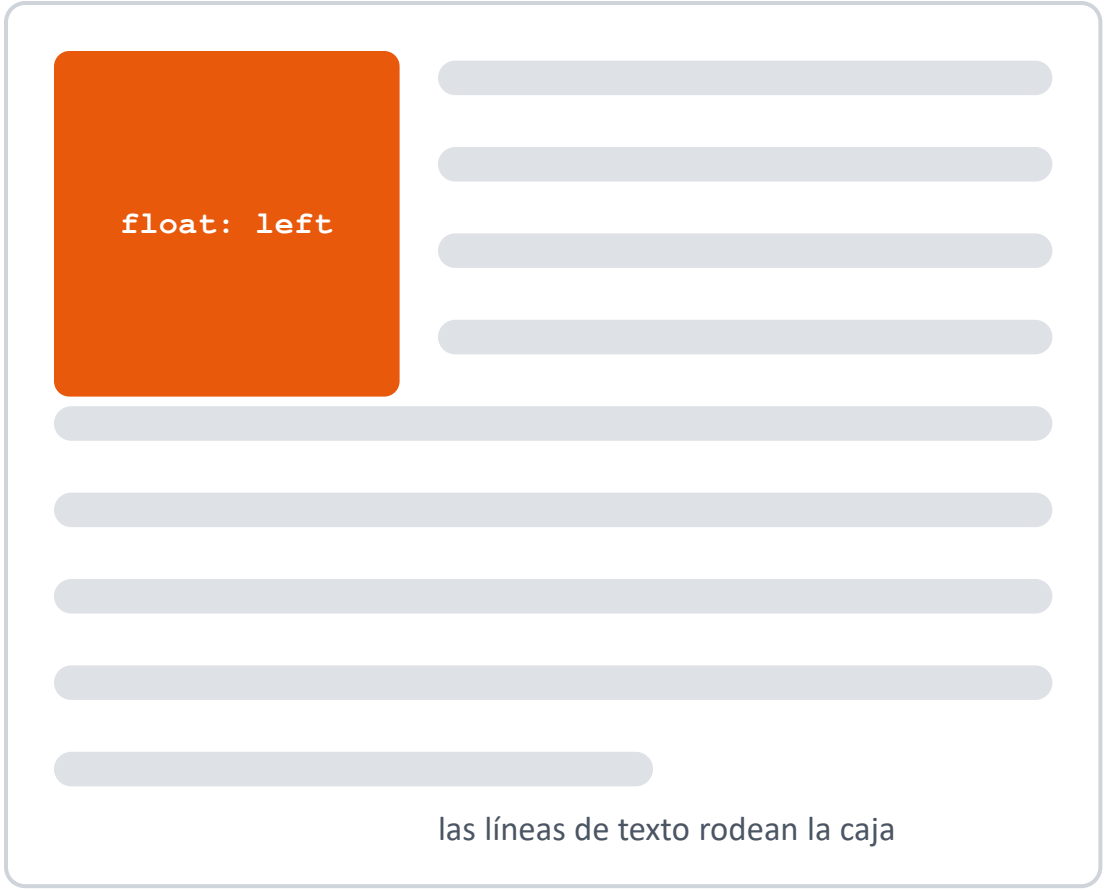
	block	inline	inline-block
¿Empieza en una línea nueva?	Sí	No	No
¿Respeta width y height?	Sí	No	Sí
¿Respeta margin arriba/abajo?	Sí	No	Sí
Ancho por defecto	todo el disponible	su contenido	su contenido
Etiquetas típicas	`div, p, h1, section`	`a, span, strong`	`button, img*`

* **`img`** y **`button`** son técnicamente inline pero se comportan como inline-block. Y **`display: none`** hace desaparecer el elemento sin dejar hueco.

float: el texto que rodea a una imagen

```css
.foto {
  float: left;  /* left | right | none */
  width: 150px;
  height: 150px;
  margin-right: 20px;  /* separa el texto */
}
```

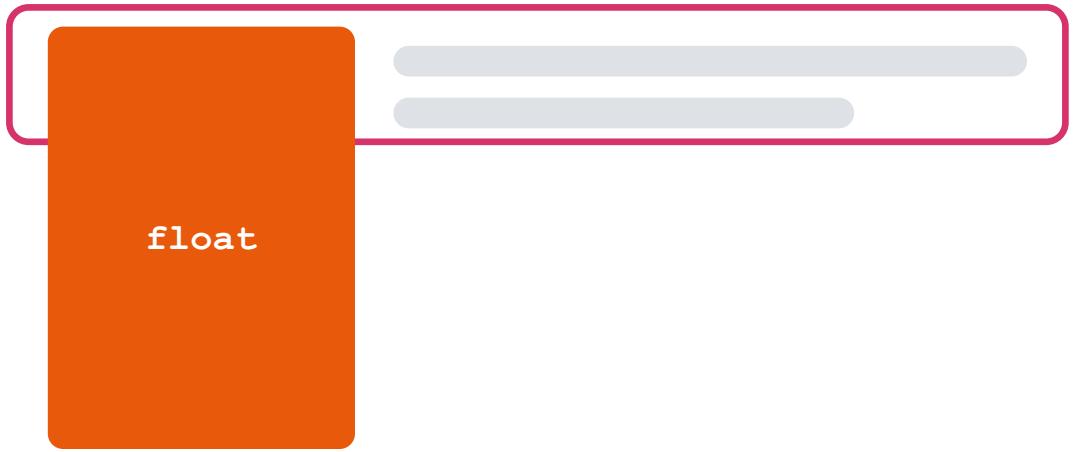
- El elemento **sale del flujo normal** y se pega a un lado.
- El texto que viene después **lo rodea**.
- Valores: `left`, `right`, `none` (por defecto) e `inherit`.



Ejemplo 02 · 02-float.html

Por qué ya no maquetamos con float

Contenedor con un float dentro



El contenedor **no ve** al float y se queda sin altura: la caja naranja se sale.

Solución moderna




```
.contenedor { display: flow-root; }
```

- Con float, columnas de igual altura, centrado vertical o repartir el ancho eran trucos frágiles. **Flexbox y Grid resuelven todo eso de forma directa.**

Ejemplo 02 · 02-float.html

02

Flexbox

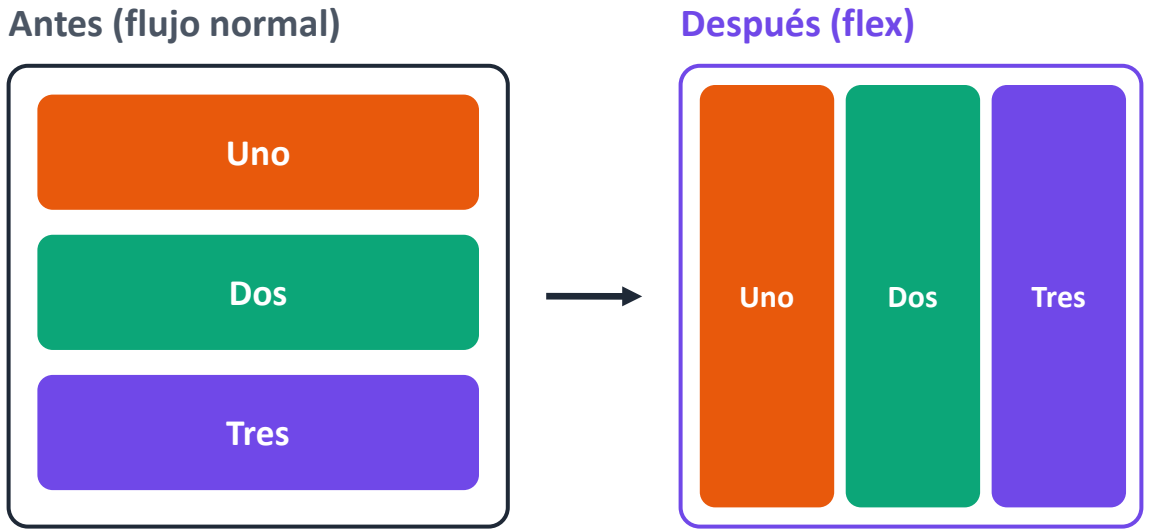
Colocar elementos en una fila o en una columna (1 dimensión)

Una línea en el padre y los hijos se ponen en fila

```html
<section>
  <article>Uno</article>
  <article>Dos</article>
  <article>Tres</article>
</section>
```

```css
section { display: flex; }
```

Se pone en el **padre**. Sus hijos directos pasan a ser **elementos flex**.



Fíjate: en flex los hijos se **estiran a la altura** del contenedor y quedan todos iguales. Con float eso era casi imposible.

Ejemplo 03 · 03-flex-direction.html

Contenedor flex y elementos flex

```
.contenedor { display: flex; }
```



Un elemento puede ser **hijo flex y contenedor flex a la vez**: así se anidan las barras, tarjetas y menús.

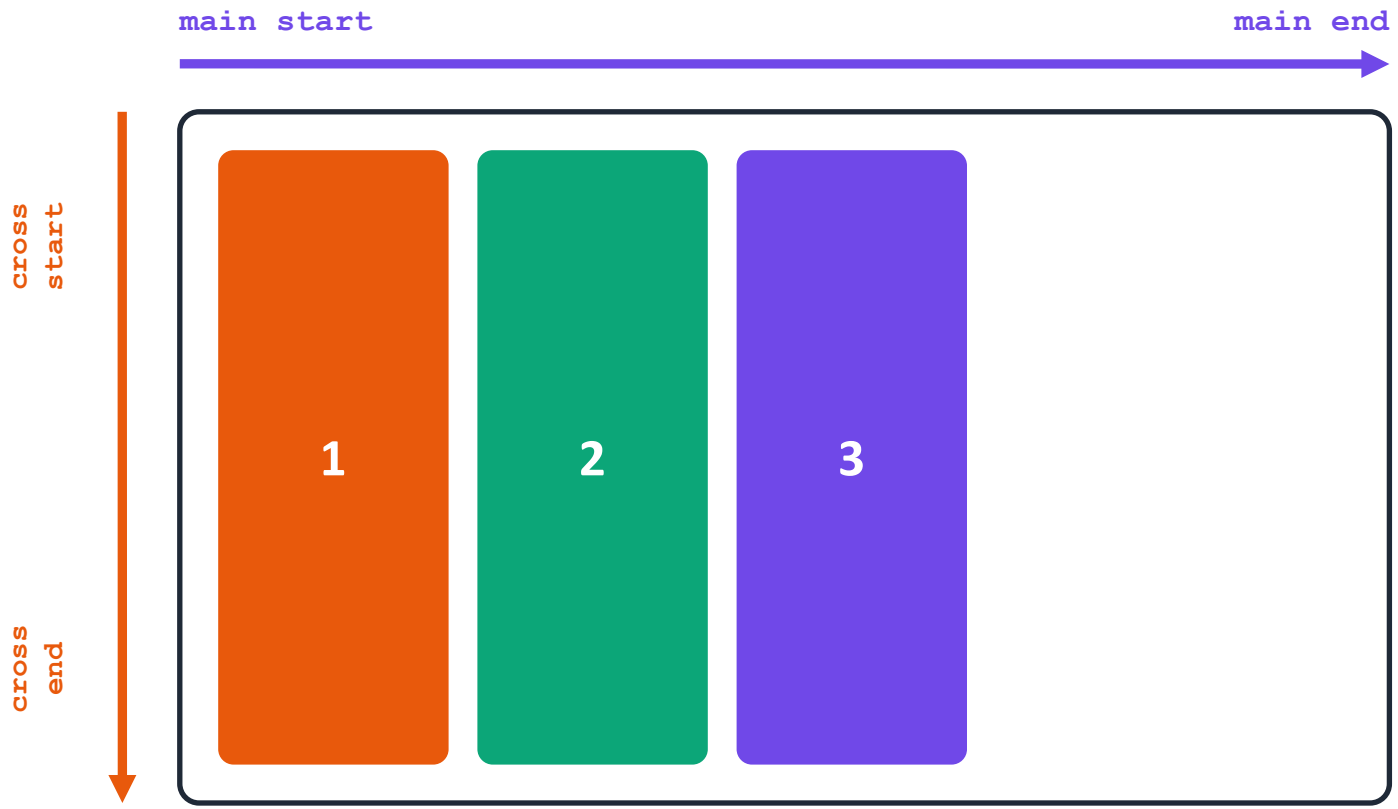
En el contenedor

- `display`
- `flex-direction`
- `justify-content`
- `align-items`
- `flex-wrap`
- `gap`

En los hijos

- `flex`
- `flex-grow`
- `flex-shrink`
- `flex-basis`
- `align-self`
- `order`

Los dos ejes de Flexbox



Eje principal (main axis)

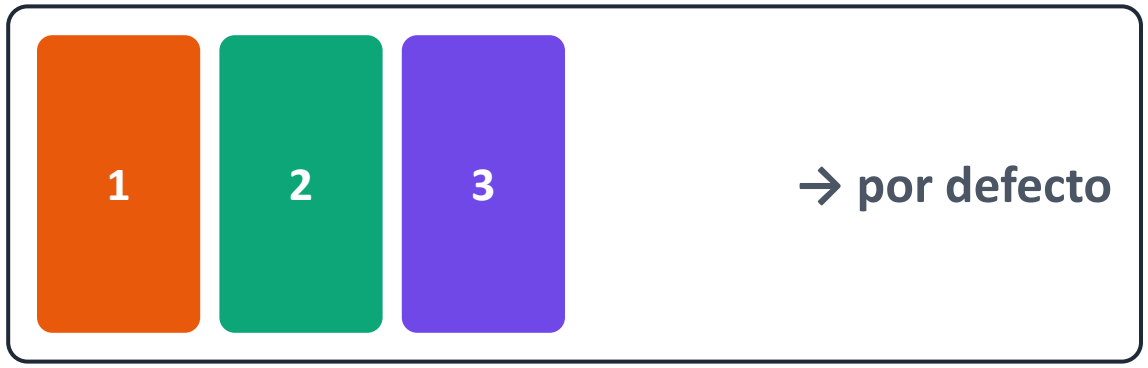
La dirección en la que se colocan los hijos. Con `row`, horizontal. Lo controla `justify-content`.

Eje transversal (cross axis)

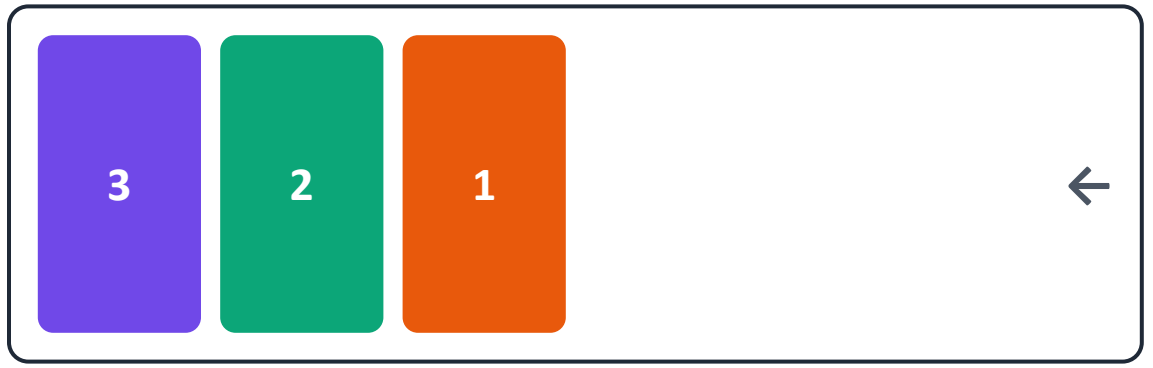
El perpendicular al principal. Con `row`, vertical. Lo controla `align-items`.

flex-direction: hacia dónde va la fila

`flex-direction: row;`



`flex-direction: row-reverse;`



`flex-direction: column;`



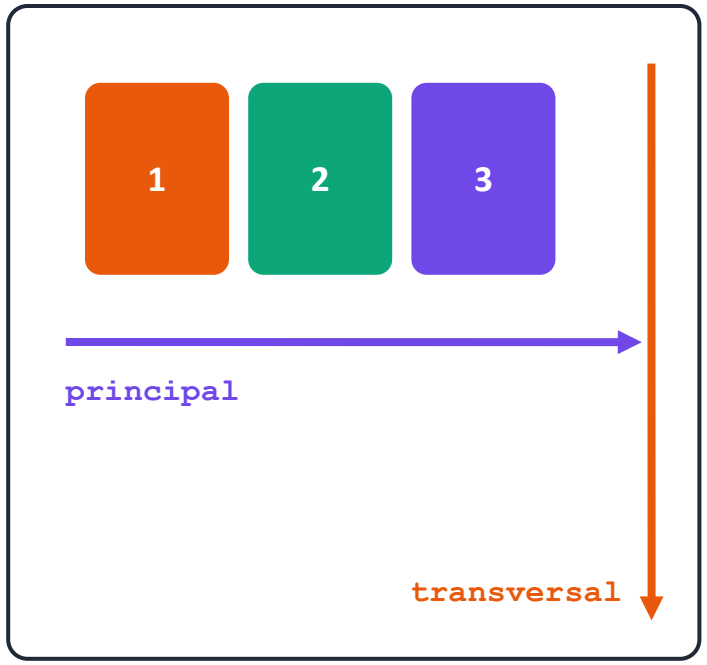
`flex-direction: column-reverse;`



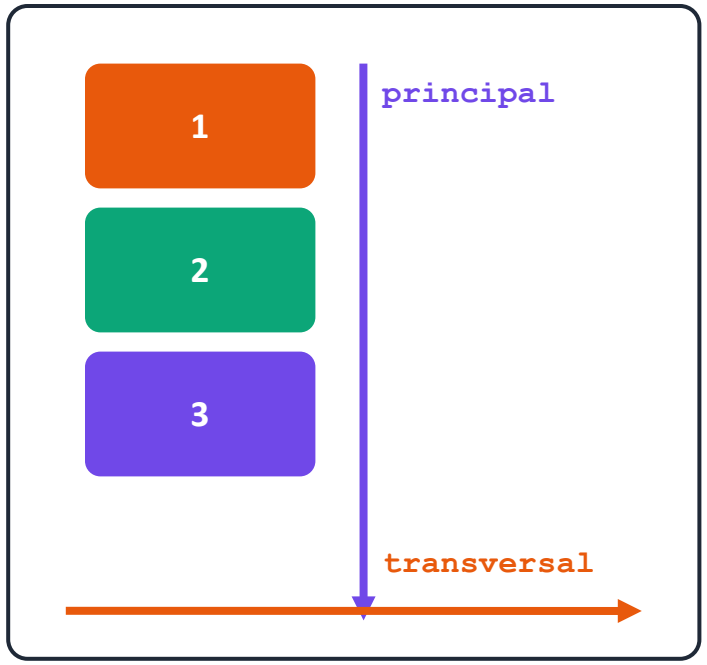
Ejemplo 03 · 03-flex-direction.html

Cuidado: con column los ejes giran

`flex-direction: row;`



`flex-direction: column;`



- Con `row`: `justify-content` mueve en **horizontal** y `align-items` en **vertical**.
- Con `column`: **al revés**. `justify-content: center` centra en vertical.
- Regla para no liarse: piensa en **principal / transversal**, no en horizontal / vertical.

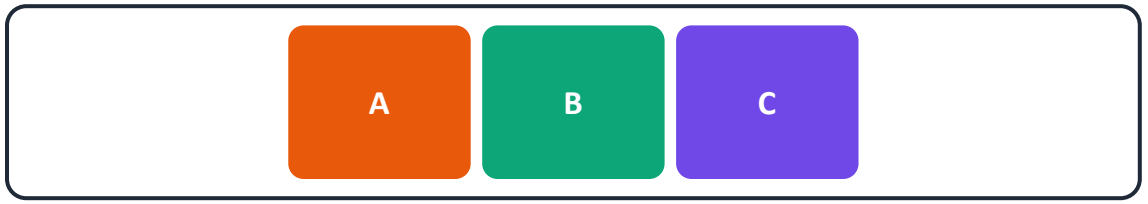
Ejemplo 08 · 08-navbar-flex.html

justify-content: repartir el espacio sobrante en la fila

`justify-content: flex-start;`



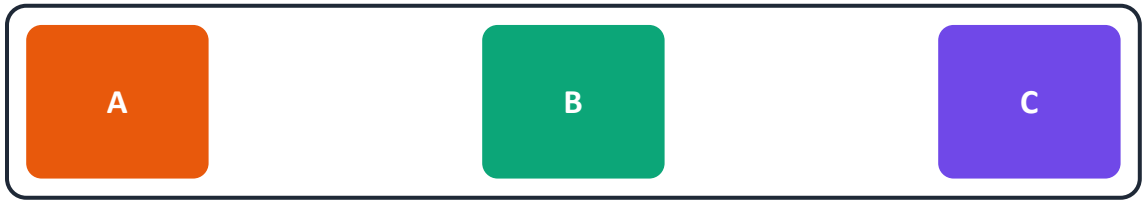
`justify-content: center;`



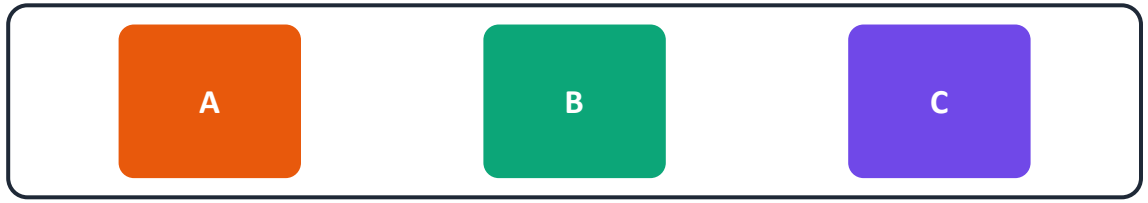
`justify-content: flex-end;`



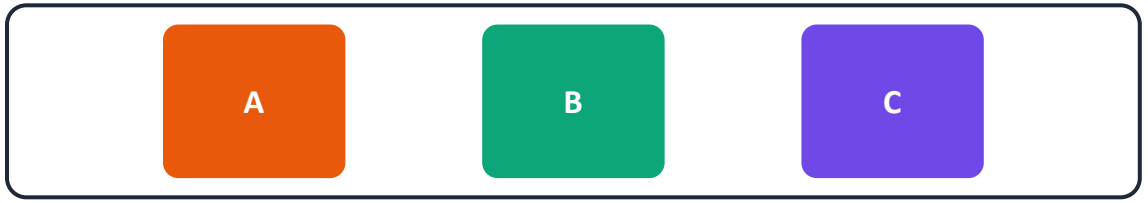
`justify-content: space-between;`



`justify-content: space-around;`



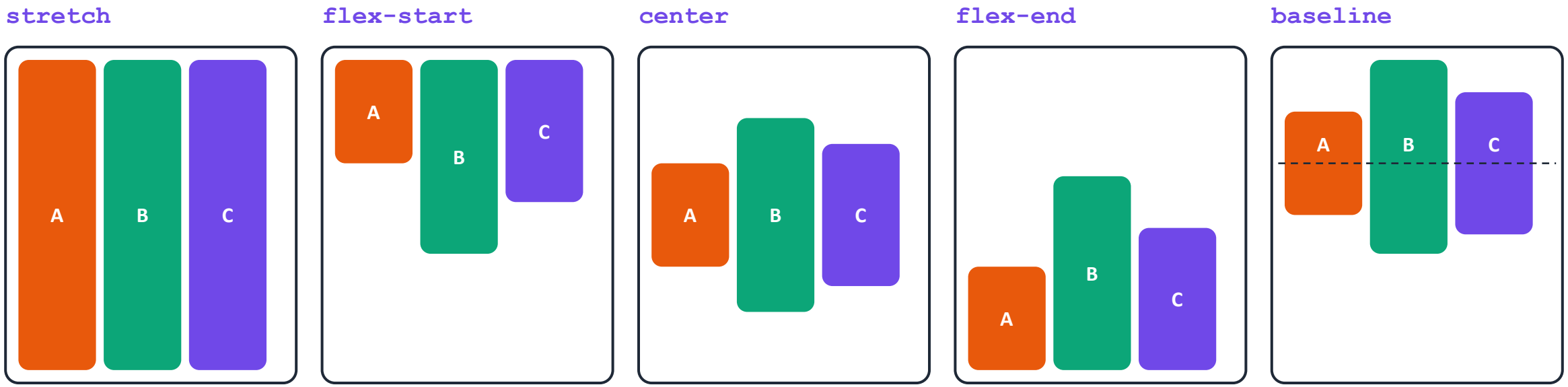
`justify-content: space-evenly;`



Solo se nota si **sobra espacio** en la fila. Valor por defecto: `flex-start`.

Ejemplo 04 · 04-justify-content.html

align-items: colocar en el eje transversal




```css
.contenedor {
  display: flex;
  align-items: center;
  height: 230px;  /* sin altura no se nota */
}
```

- `stretch` (defecto): todos ocupan la altura completa.
- `baseline`: alinea la **línea del texto**, aunque las cajas tengan distinto padding.

Ejemplo 05 · 05-align-items-centrado.html

El centrado perfecto en 3 líneas

```css
.escenario {
  display: flex;
  justify-content: center;   /* centro horizontal */
  align-items: center;   /* centro vertical */
  height: 300px;
}
```

- Todo va en el **padre**. El hijo no necesita nada.
- Funciona con cualquier tamaño de hijo, **sin calcular** márgenes ni posiciones.

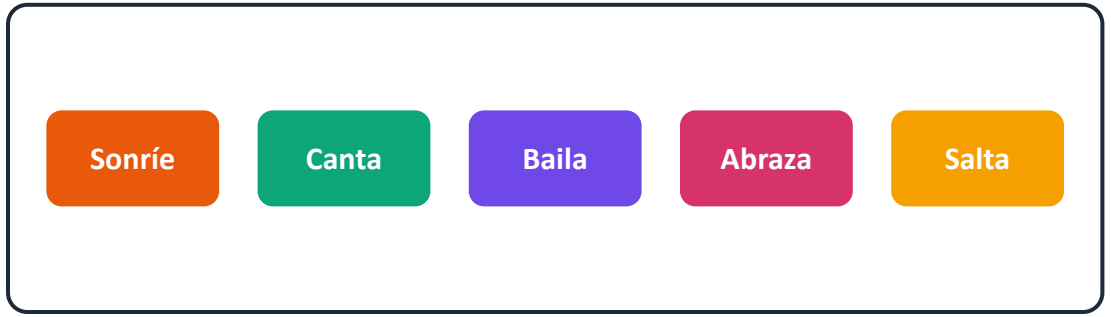


Ejemplo 05 · 05-align-items-centrado.html

Ejemplo del temario: fila de botones

```html
<div>
  <button>Sonríe</button> <button>Canta</button>
  <button>Baila</button> <button>Abraza</button>
  <button>Salta</button>
</div>
```

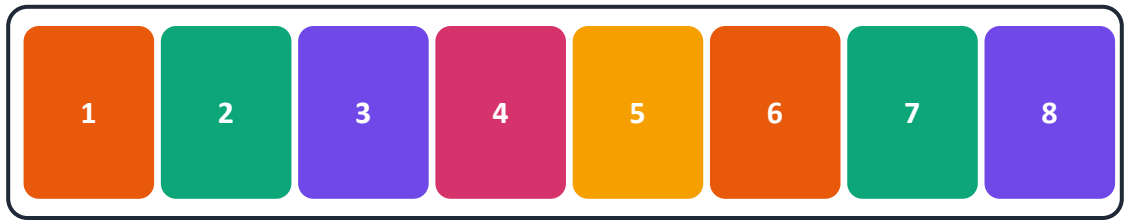
```css
div {
  height: 100px;
  border: 1px solid black;
  display: flex;
  align-items: center;  /* vertical */
  justify-content: space-around;  /* horizontal */
}
```



space-around deja el mismo espacio a cada lado de cada botón; **center** los pone a media altura de los 100px.

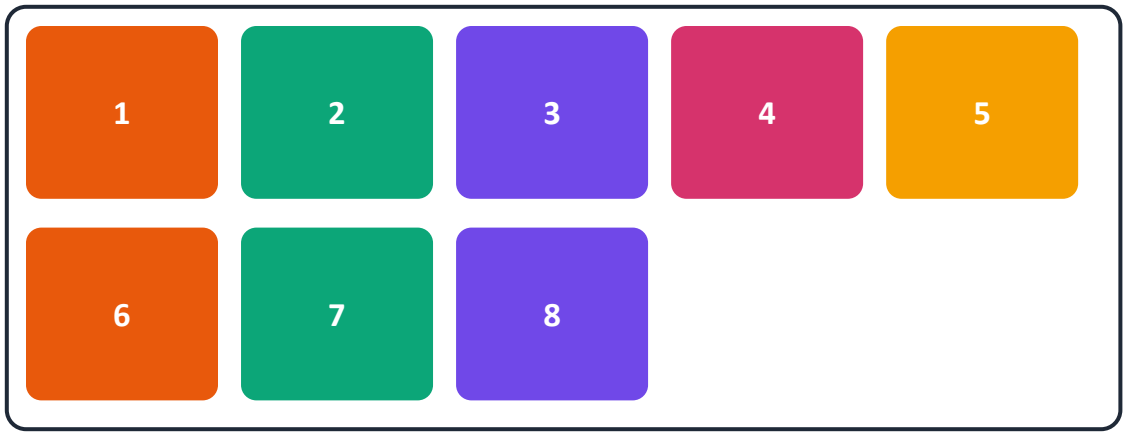
flex-wrap: cuando no caben en una fila

`flex-wrap: nowrap;` (por defecto)



Se **encogen** para caber en una sola fila.

`flex-wrap: wrap;`

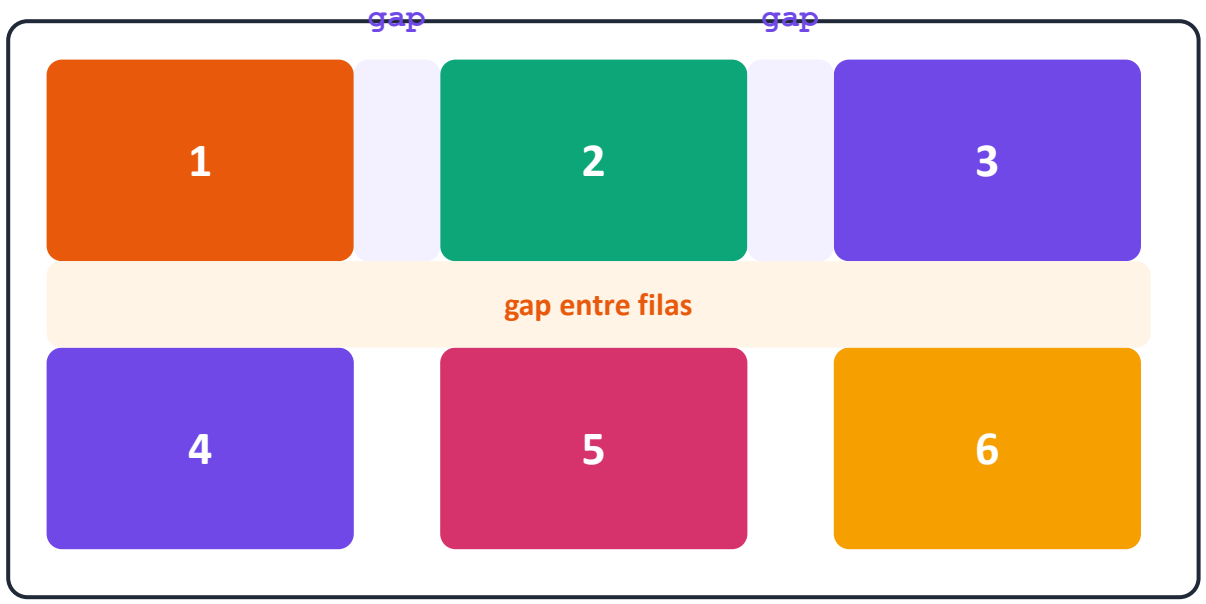


Mantienen su ancho y **bajan a la siguiente línea**.

```
.galeria { display: flex; flex-wrap: wrap; gap:
14px; }
.foto { width: 150px; }
```

Ejemplo 06 · 06-flex-wrap-gap.html

gap: separar sin pelearse con márgenes




```css
gap: 20px;  /* filas y columnas */
gap: 20px 10px;  /* filas | columnas */
row-gap: 20px;
column-gap: 10px;
```

- Solo crea espacio **entre** elementos, no en los bordes del contenedor.
- Sustituye a los `margin` en los hijos (y a sus problemas con el último elemento).
- La misma propiedad sirve en **Grid**.

Ejemplo 06 · 06-flex-wrap-gap.html

flex: 1, cuánto crece cada hijo

```
.item { flex: 1; }
```

es una abreviatura de tres propiedades:

`flex-grow: 1`	Cuánto **crece** si sobra espacio (proporción)
`flex-shrink: 1`	Cuánto **encoge** si falta espacio
`flex-basis: 0`	Tamaño **de partida** antes de repartir

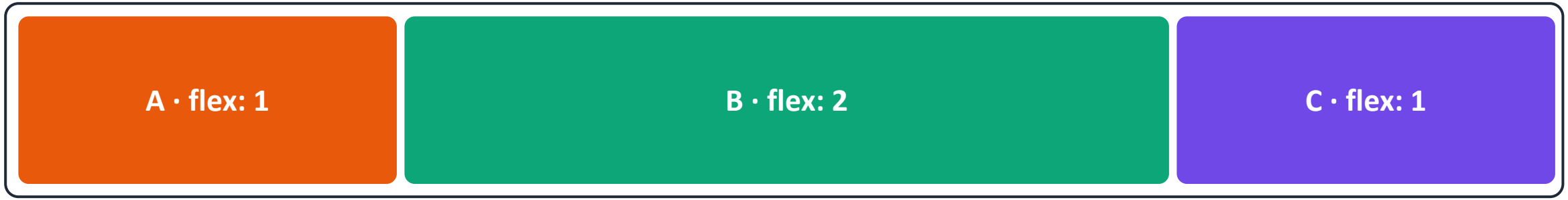
Los tres con flex: 1 → iguales



El espacio libre se reparte **en partes**. Con tres hijos a `flex: 1`, cada uno se lleva 1/3, sin calcular porcentajes y sin importar el ancho de la pantalla.

Ejemplo 07 · 07-flex-crecer.html

Proporciones: flex: 1 / 2 / 1




```
.a { flex: 1; }
.b { flex: 2; }  /* el doble que A y C */
.c { flex: 1; }
```

- Total = 1 + 2 + 1 = **4 partes**: A y C se llevan 1/4, B se lleva 2/4.
- `flex: 0 0 auto` = no crecer ni encoger: mide lo que su contenido.

Ejemplo 07 · 07-flex-crecer.html

Patrón real: menú fijo + contenido flexible




```
.app { display: flex; }
.menu { width: 200px; flex-shrink: 0; }
.contenido { flex: 1; }
```

- El menú tiene **ancho fijo** y no encoge.
- El contenido se queda con **todo lo que sobra**, sea el ancho que sea.

Ejemplo 07 · 07-flex-crecer.html

margin-left: auto, empujar al otro extremo



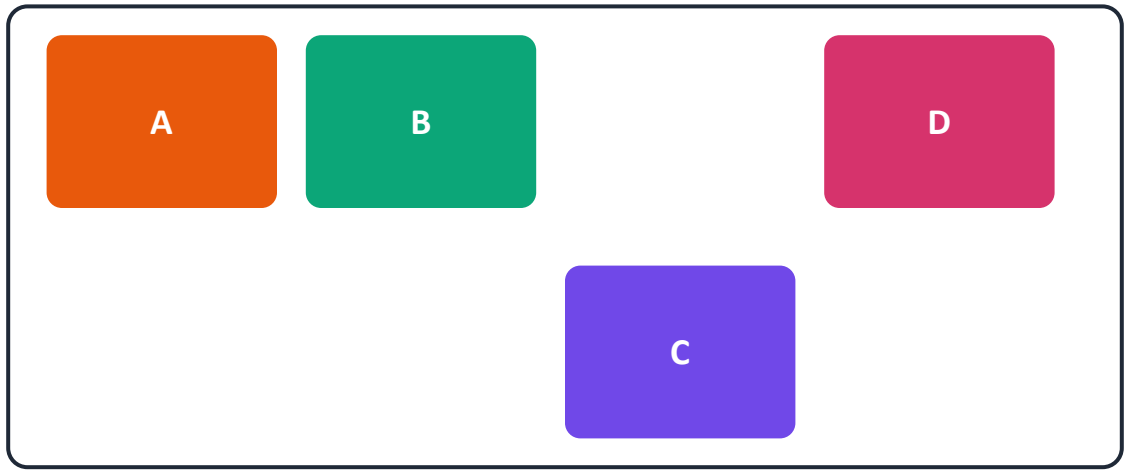

```css
.nav {
  display: flex;
  align-items: center;  /* todo a media altura */
  gap: 24px;
}
.boton { margin-left: auto; }
```

- Un margen `auto` en flex **se come todo el espacio libre**.
- Resultado: logo y enlaces a la izquierda, botón a la derecha, con una sola línea.
- `align-items: center` alinea logo, enlaces y botón aunque tengan tamaños distintos.

Ejemplo 08 · 08-navbar-flex.html

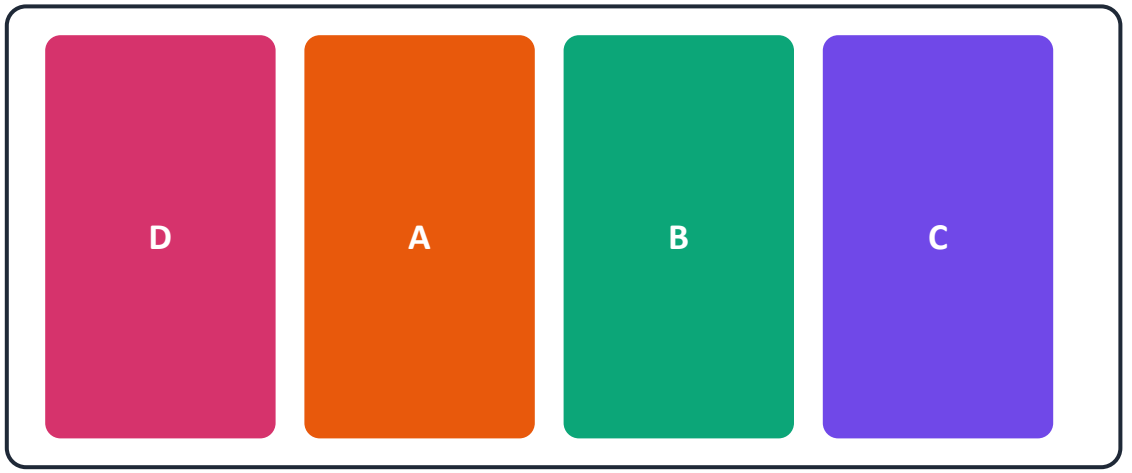
Excepciones en un solo hijo: align-self y order

```
.c { align-self: flex-end; }
```



Todos usan `align-items: flex-start` del padre, pero C **se lo salta** y se va abajo.

```
.d { order: -1; }
```



`order` cambia la posición **visual** sin tocar el HTML. Por defecto todos valen 0; los valores menores van antes.

Ojo con `order`: los lectores de pantalla y el tabulador siguen el orden del HTML.

Chuleta de Flexbox

En el contenedor (padre)	Para qué	En los hijos	Para qué
`display: flex`	Activa Flexbox	`flex: 1`	Crecer para llenar el hueco
`flex-direction`	row | column (+ -reverse)	`flex: 2`	Doble de espacio que flex: 1
`justify-content`	Repartir en el eje principal	`flex-shrink: 0`	No encoger nunca
`align-items`	Colocar en el eje transversal	`align-self`	align-items solo para ese hijo
`flex-wrap: wrap`	Saltar de línea si no caben	`order`	Cambiar el orden visual
`gap`	Espacio entre hijos	`margin-left: auto`	Empujar al otro extremo

Centrado perfecto: `display: flex; justify-content: center; align-items: center;`

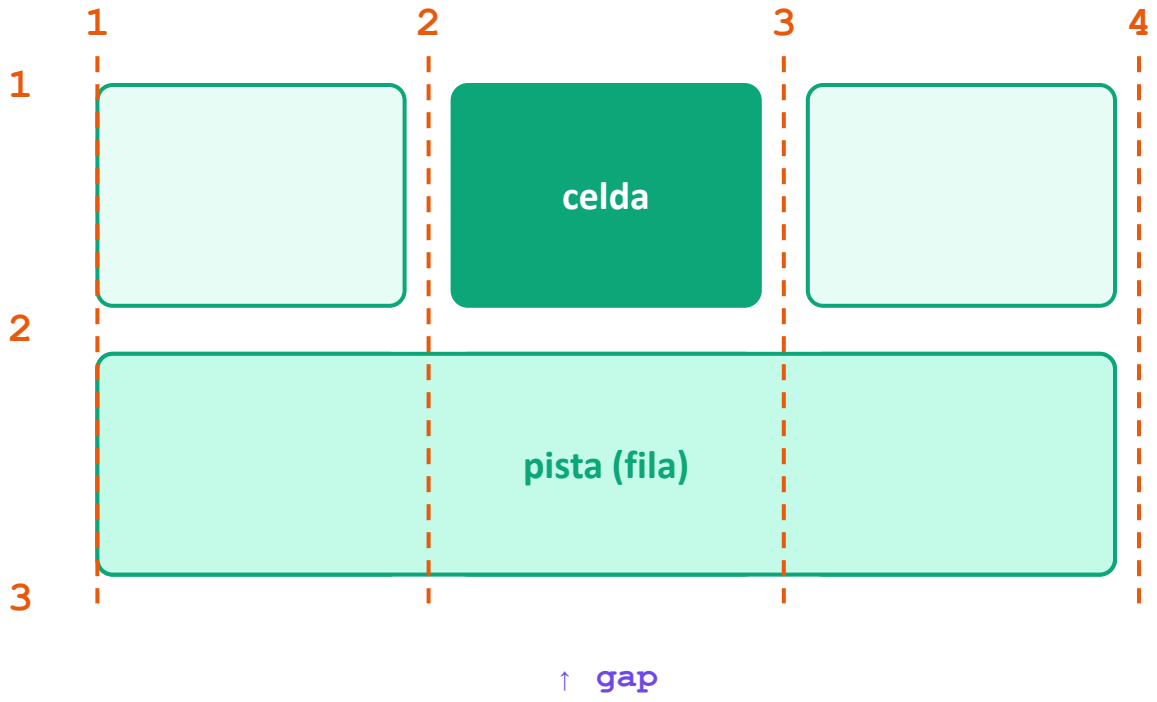


03

Grid

Filas y columnas a la vez (2 dimensiones)

Vocabulario de Grid



- **Grid** = cuadrícula de líneas horizontales y verticales.
- **Celda**: el hueco entre cuatro líneas.
- **Pista** (track): una fila o una columna completa.
- **Líneas**: se numeran desde 1. Con 3 columnas hay 4 líneas.
- **gap** (gutter): el hueco entre pistas.

display: grid, y le decimos las columnas

```html
<div class="container">
  <div>Uno</div>   <div>Dos</div>
  <div>Tres</div>  <div>Cuatro</div>
  <div>Cinco</div> <div>Seis</div> <div>Siete</div>
</div>
```

```css
.container {
  display: grid;  /* solo esto: 1 columna */
}
```

- Igual que en flex: va en el **padre** y los hijos directos pasan a ser **elementos grid**.

Resultado: una sola columna

Uno

Dos

Tres

Cuatro

Cinco

Seis

Siete

Ejemplo 09 · 09-grid-columnas.html

grid-template-columns con píxeles

```css
.container {
  display: grid;
  grid-template-columns: 200px 200px 200px;
}
```

- Tres columnas **fijas** de 200px.
- No se adaptan al ancho de la pantalla.



Ejemplo 09 · 09-grid-columnas.html

La unidad fr: fracciones del espacio libre

`grid-template-columns: 1fr 1fr 1fr;` → 3 partes iguales



`grid-template-columns: 2fr 1fr 1fr;` → 4 partes: 2 + 1 + 1



- `fr` reparte el espacio **disponible** (después de quitar los `gap`). Se adapta solo a cualquier pantalla.
- Se puede mezclar: `250px 1fr` = columna fija + columna que ocupa el resto.

Ejemplo 09 · 09-grid-columnas.html

repeat() y gap: el CSS del temario

```css
.container {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 20px;
}
```

- `repeat(3, 1fr)` es lo mismo que `1fr 1fr 1fr`.
- `repeat(4, 1fr)`: cuatro columnas cambiando un número.
- Los hijos se colocan **solos**, en orden, rellenando fila a fila.
- Si no caben, grid **crea filas nuevas** automáticamente.



← 1fr → gap ← 1fr → gap ← 1fr →

Ejemplo 09 · 09-grid-columnas.html

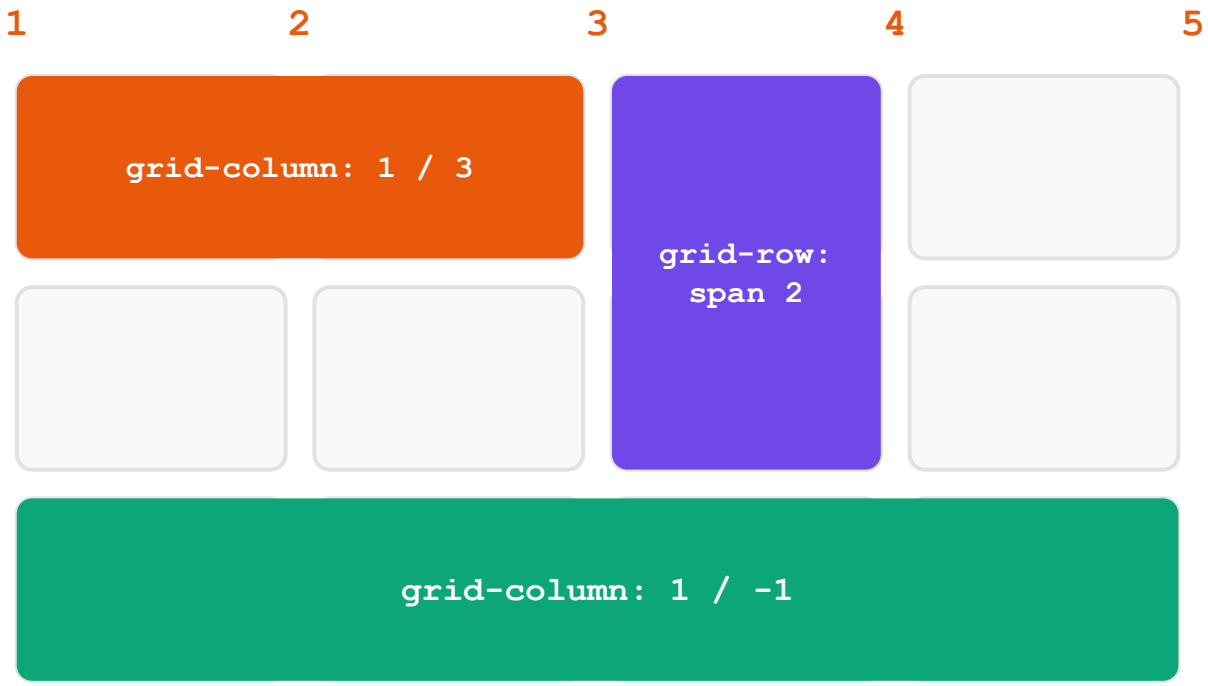
Filas: grid-template-rows y grid-auto-rows

```css
.container {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  grid-auto-rows: 100px;
  gap: 20px;
}
```

- `grid-template-rows`: filas **explícitas**, las que tú defines (`100px 200px`).
- `grid-auto-rows`: altura de las filas **implícitas**, las que grid crea cuando faltan.
- Truco: `grid-auto-rows: minmax(100px, auto)` = al menos 100px, más si el contenido lo pide.



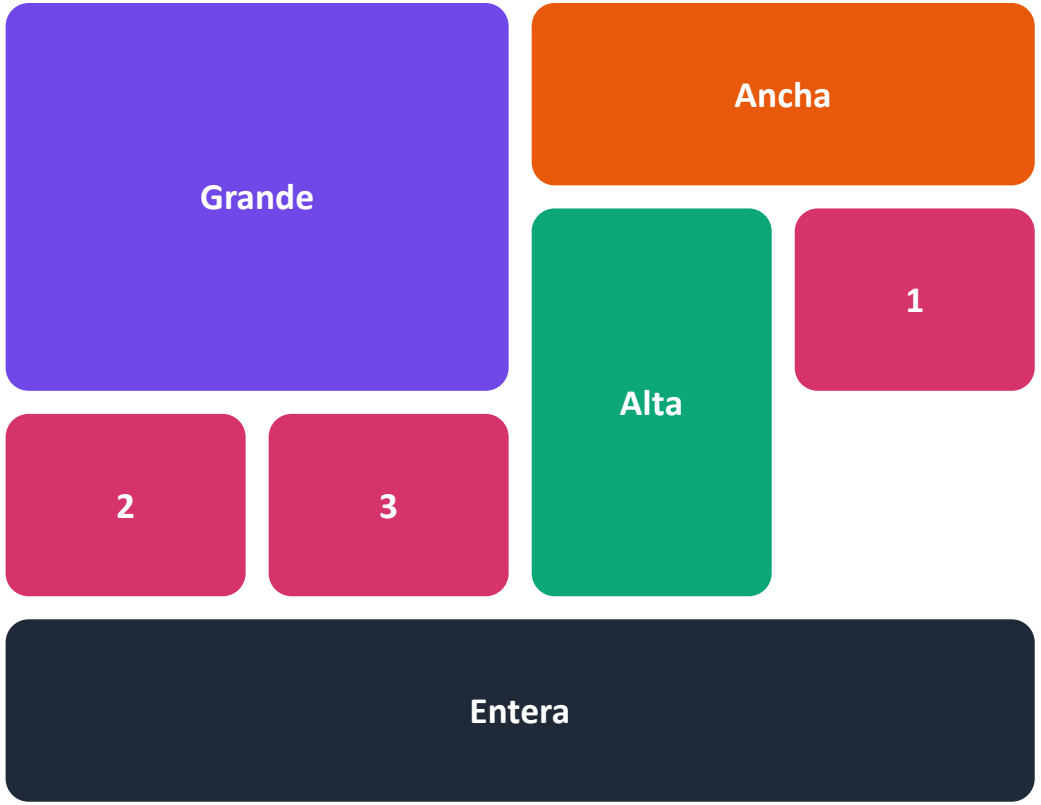
Colocar por líneas: grid-column y grid-row



- `grid-column: 1 / 3`: de la **línea 1 a la 3** (ocupa 2 columnas).
- `grid-row: span 2`: ocupa **2 filas** desde donde le toque.
- `grid-column: 1 / -1`: de la primera a la **última** línea. Ancho completo siempre.
- Los demás elementos se recolocan solos en los huecos libres.

Caso real: cuadrícula tipo bento con span

```css
.bento {
  display: grid;
  grid-template-columns: repeat(4, 1fr);
  grid-auto-rows: 110px;
  gap: 14px;
}
.grande { grid-column: span 2; grid-row: span 2; }
.ancha { grid-column: span 2; }
.alta { grid-row: span 2; }
.entera { grid-column: 1 / -1; }
```

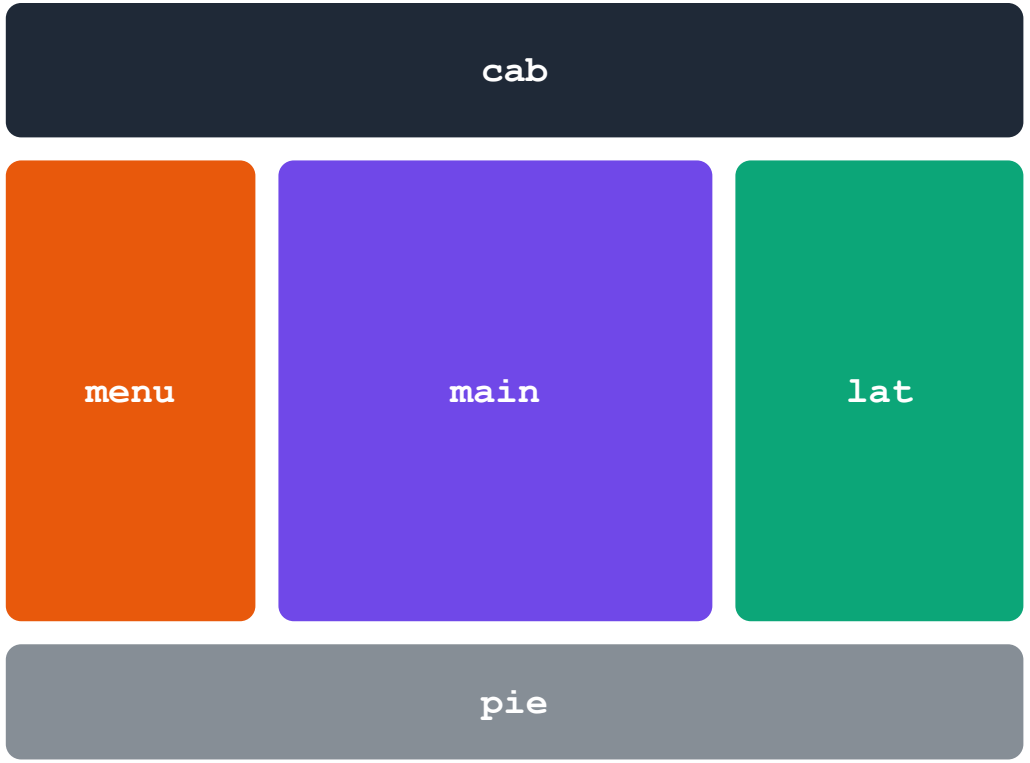


- Solo decimos **cuánto ocupa** cada caja; grid decide **dónde** va.

Ejemplo 10 · 10-grid-span-bento.html

grid-template-areas: dibujar la página con texto

```
.pagina {
  display: grid;
  grid-template-columns: 180px 1fr 200px;
  grid-template-rows: 70px 1fr 60px;
  grid-template-areas:
    "cab   cab   cab"
    "menu  main  lat"
    "pie   pie   pie";
  gap: 12px;
}
header { grid-area: cab; }
nav { grid-area: menu; }
main { grid-area: main; }
```



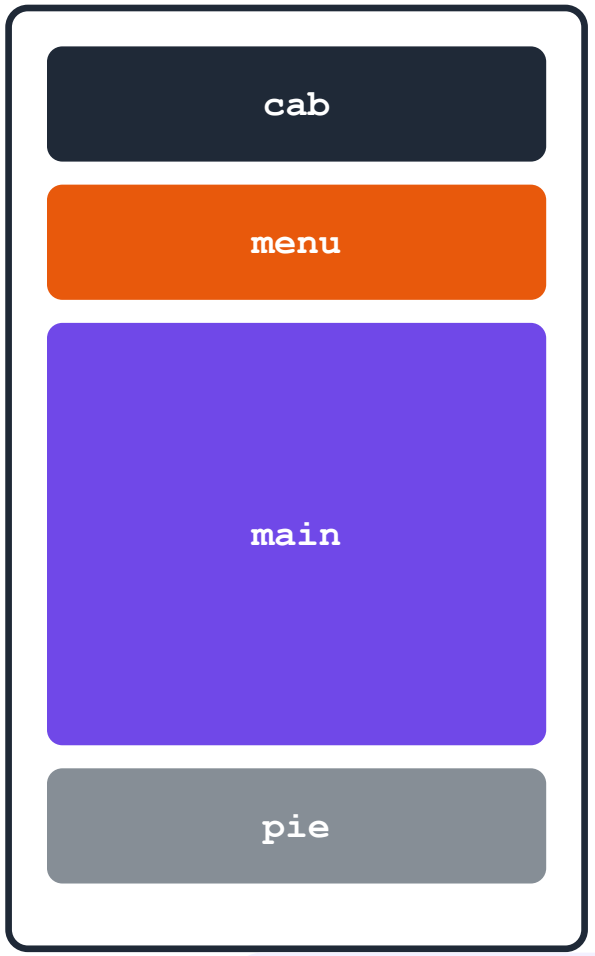
- Cada cadena entre comillas es **una fila**; repetir un nombre hace que la zona **ocupe varias celdas** (tienen que ser rectángulos).

Ejemplo 11 · 11-grid-areas.html

Mismo HTML, otro dibujo: versión móvil

```
@media (max-width: 700px) {
  .pagina {
    grid-template-columns: 1fr;
    grid-template-areas:
      "cab"
      "menu"
      "main"
      "pie";
  }
}
```

- No se toca el HTML ni los `grid-area` de los hijos.
- Solo se **redibuja** el mapa de zonas.



Ejemplo 11 · 11-grid-areas.html

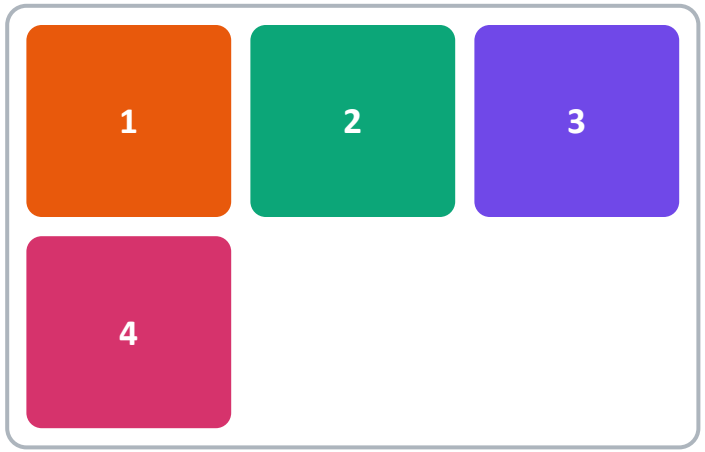
Galería responsive sin media queries

```
.galeria { display: grid; grid-template-columns: repeat(auto-fit, minmax(200px, 1fr)); gap: 16px; }
```

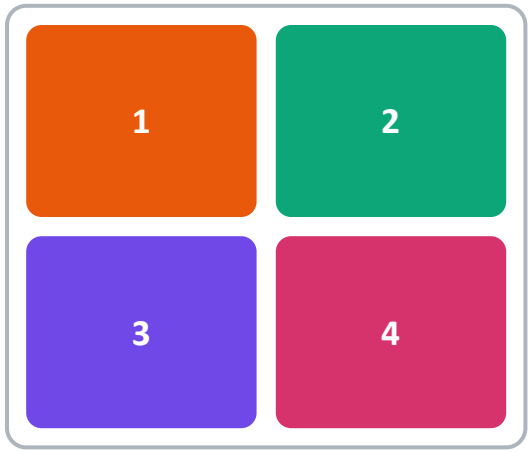
Pantalla ancha: 4 columnas



Tablet: 3 columnas



Móvil: 2 columnas



- `minmax(200px, 1fr)`: cada columna mide **al menos 200px**, como mucho 1fr. `auto-fit`: **todas las columnas que quepan**.

Ejemplo 12 · 12-grid-responsive.html

Centrar con Grid: place-items

Flexbox (3 líneas)

```
.padre {
  display: flex;
  justify-content: center;
  align-items: center;
}
```

Grid (2 líneas)

```
.padre {
  display: grid;
  place-items: center;
}
```

centrado

Los dos sirven. Usa el que ya tengas en el padre.

Chuleta de Grid

En el contenedor (padre)	Para qué	En los hijos	Para qué
`display: grid`	Activa Grid	`grid-column: span 2`	Ocupar 2 columnas
`grid-template-columns`	Definir columnas (px, fr)	`grid-row: span 2`	Ocupar 2 filas
`repeat(3, 1fr)`	3 columnas iguales	`grid-column: 1 / -1`	Ancho completo
`grid-auto-rows`	Altura de filas automáticas	`grid-area: main`	Ir a una zona con nombre
`grid-template-areas`	Dibujar zonas con nombre	`place-self: center`	Centrar solo ese hijo
`repeat(auto-fit, minmax())`	Columnas responsive		
`gap / place-items`	Huecos / centrar hijos		

04

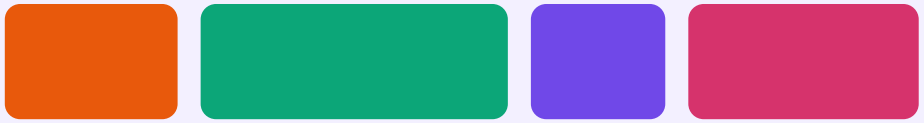
¿Flex o Grid?

Cuándo usar cada uno y cómo se combinan

Flexbox o Grid: la regla práctica

Flexbox

1 dimensión: una fila o una columna.



- El **contenido** decide el tamaño.
- Barras de navegación, botones, alinear iconos con texto, centrar.

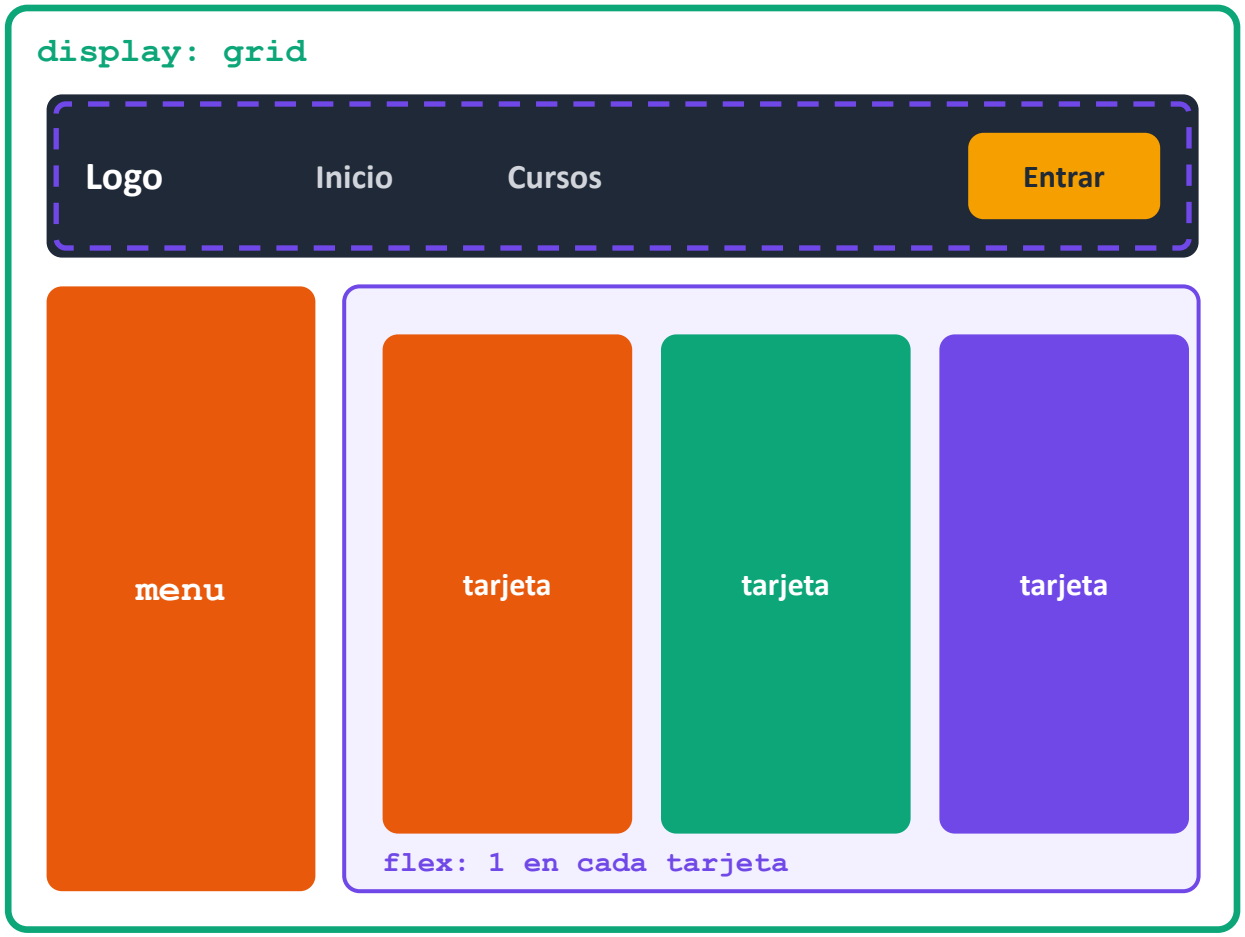
Grid

2 dimensiones: filas y columnas a la vez.



- La **estructura** decide el tamaño.
- Layout de página, galerías, paneles, formularios en columnas.

Lo normal es combinarlos




```
.pagina { display: grid;
          grid-template-areas: ...; }
.cabecera { display: flex;
            align-items: center; }
.tarjetas { display: flex; gap: 16px; }
```

- **Grid** coloca las grandes zonas de la página.
- **Flex** alinea lo que hay dentro de cada zona (marcas moradas).

Los 6 errores que más vais a cometer

1 display: flex en el hijo

Va en el **padre**. Los hijos solo reciben `flex`, `align-self`, `grid-column`...

2 align-items sin altura

Si el contenedor no tiene `height`, no hay espacio vertical que repartir.

3 Olvidar que column gira los ejes

Con `column`, `justify-content` actúa en **vertical**.

4 Maquetar con float

Solo para texto alrededor de imágenes. Para columnas: flex o grid.

5 Columnas en px

`200px 200px 200px` no se adapta. Usa `fr` o `minmax()`.

6 Márgenes para separar

Usa `gap`: no deja espacio extra en los bordes.

Reto de 5 minutos

Construid esta página desde cero:

1. Cabecera con logo a la izquierda y botón a la derecha

   `display: flex + margin-left: auto`

2. Bloque con un título centrado en horizontal y vertical

   `justify-content + align-items: center`

3. Galería de 6 tarjetas que se adapte al ancho

   `grid + repeat(auto-fit, minmax(150px, 1fr))`

4. Separación entre todo sin usar margin

   `gap`



Para llevarse

display

Todo empieza aquí: block, inline, flex o grid deciden cómo se colocan los hijos.

Flexbox

Una dimensión. Ejes principal y transversal: justify-content y align-items. flex: 1 para repartir.

Grid

Dos dimensiones. Columnas con fr y repeat(), span para ocupar varias celdas, areas para dibujar la página.

Repasad en casa con `ejemplos-layout-css/index.html`: cada ejemplo tiene botones, el CSS exacto y la explicación línea a línea.

Referencia: MDN Web Docs, guías de Flexbox y CSS Grid Layout (developer.mozilla.org).